

INVESTMENT INTELLIGENCE PLATFORM

Uygulamanın çalışma modeli, veri akışı ve Kubernetes mimarisi

Mimari inceleme raporu

Bu rapor, mevcut repository içindeki backend, frontend, Helm, ArgoCD, PostgreSQL ve n8n yapılandırmalarına dayanır. Mevcut Phase 1 davranışı ile ileride eklenmesi planlanan analiz ve yapay zekâ katmanları ayrı değerlendirilmiştir.

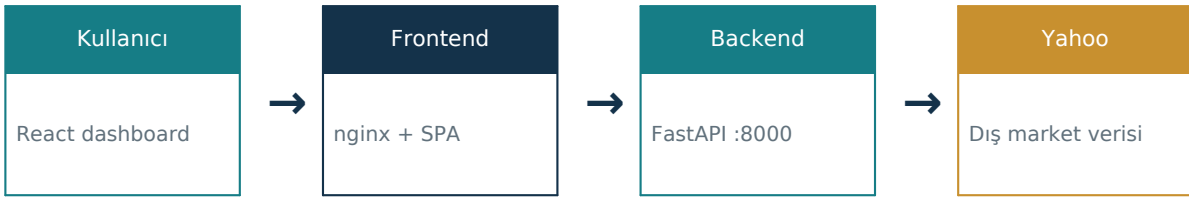
Rapor kapsamı	Mevcut kod ve deployment tanımları
Uygulama aşaması	Phase 1 — temel platform ve market görünümü
Çalışma ortamı	Kubernetes, Helm, ArgoCD, Docker, PostgreSQL, n8n
Hazırlanma tarihi	17 Ağustos 2026

1. Yönetici özeti

Platform; tarayıcıda çalışan React/Vite arayüzünü, HTTP API sunan FastAPI backend'ini, kalıcı PostgreSQL veritabanını ve otomasyon/workflow katmanı olarak n8n'i Kubernetes üzerinde bir araya getirir. ArgoCD, Git repository içindeki deployment tanımlarını cluster'a otomatik senkronize eder; Helm ise bileşenlerin Deployment, Service, Secret, ConfigMap, PVC, Ingress ve NetworkPolicy kaynaklarını üretir.

Bugünkü gerçek çalışma kapsamı Phase 1'dir. Backend health/readiness uç noktalarını ve Yahoo Finance veya mock provider üzerinden market overview/history uç noktalarını sunar. Teknik indikatörler, sinyaller, portföy, AI analist ve alarm akışları repository dokümanında sonraki fazlar olarak tanımlanmıştır; bunlar henüz mevcut kodda tamamlanmış özellikler değildir.

En kısa uçtan uca akış şöyledir:



Backend'in veritabanı bağlantısı hazırdır ve readiness kontrolünde kullanılır; ancak Phase 1 market endpoint'leri veriyi PostgreSQL'e kaydetmez. PostgreSQL'in asıl işlevi sonraki fazlarda kalıcı market zaman serileri, haberler, sinyaller, portföy ve AI raporları için merkezi state store olmaktır. n8n ise hesaplamanın sahibi değil, zamanlama, HTTP çağrılarını, dallanma, dış servisler, bildirimler ve rapor dağıtımını için orkestrasyon katmanıdır.

2. Bileşenler ve sorumluluk sınırları

Bileşen	Bugünkü görevi	Kubernetes karşılığı
React + Vite	Dashboard, durum sayfası, market verisini gösterme	Frontend Deployment + NodePort/Ingress
nginx	Statik dosya sunumu, /api proxy, SPA fallback, /health	Frontend container
FastAPI + Uvicorn	HTTP API, CORS, health/readiness, market provider çağrıları	Backend Deployment + ClusterIP
SQLAlchemy async	PostgreSQL async engine/session altyapısı	Backend process içi kütüphane
PostgreSQL	Kalıcı ilişkisel veri; n8n için ayrı database	Bitnami StatefulSet + PVC veya ArgoCD chart
n8n	Workflow scheduling ve dış sistem entegrasyonu	Deployment + PVC + ClusterIP
Helm	Kubernetes manifest templating ve environment override	Chart
ArgoCD	GitOps sync, self-heal, prune, retry	ArgoCD Application CR'ları

Repository'deki iki deployment modeli

- Tek chart kurulumu: values.yaml içinde backend, frontend, n8n ve PostgreSQL aynı Helm release ile açılabilir.

- Split-app kurulumu: mevcut ArgoCD manifestleri dört ayrı Application kullanıyor.

Backend/frontend/n8n chart'ı PostgreSQL'i kapatıyor; PostgreSQL ayrı Bitnami chart olarak kuruluyor.

Split-app modelinde global.databaseHost, global.backendServiceHost ve global.postgresSecretName değerleri bağımsız release'lerin gerçek Service/Secret isimleriyle eşleşmelidir. Bu isimler Kubernetes DNS ve Secret referansları açısından kritik çalışma sözleşmesidir.

3. Kullanıcı isteği nasıl çalışır?

Production'da varsayılan giriş noktası Ingress'tir. values-prod.yaml, aynı host üzerinde '/' yolunu frontend'e, '/api' yolunu backend'e yönlendirir. Frontend container'ı ayrıca kendi nginx proxy'siyle /api çağrılarını BACKEND_HOST üzerinden backend Service'ine aktarabilir. Bu nedenle deployment biçimine göre API yolu ya Ingress üzerinden ya da frontend nginx iç proxy'si üzerinden çözülür.

Tarayıcı → https://invest.yourdomain.com/
/dashboard → frontend Service → nginx → index.html
/api/markets/overview → backend Service → FastAPI router
/api/ready → backend → PostgreSQL SELECT 1

Frontend tarafında api.ts, VITE_API_BASE_URL boşsa görece URL kullanır. Dashboard ilk açılışta /api/markets/overview çağrısı yapar, 60 saniyede bir yeniler ve manuel Refresh butonuna izin verir. Axios interceptor, backend'in detail alanını kullanıcıya taşır. Health ve readiness hooks ise React Query ile sırasıyla 60 ve 30 saniyede yenilenir.

Frontend nginx yapılandırması

- /api/ isteklerini http://\${BACKEND_HOST}:8000 adresine proxy eder.
- Bilinmeyen UI yollarını /index.html'e yönlendirerek React SPA routing'i destekler.
- /health endpoint'i Kubernetes probe'ları için statik 200 döndürür.
- Gzip, immutable asset cache ve temel güvenlik header'ları uygular.

4. Backend nasıl çalışır?

Uygulama backend/app/main.py içindeki create_app() ile oluşturulur. Pydantic Settings, environment değişkenlerini .env dosyası veya process environment içinden yükler. FastAPI; /api/docs, /api/redoc ve /api/openapi.json dokümantasyon uç noktalarını açar; health ve markets router'larını register eder.

Açılış ve kapanış yaşam döngüsü

- Startup'ta ayarlar okunur ve yapılandırılmış structlog kaydı yazılır.
- Async SQLAlchemy engine oluşturulur; SELECT 1 ile veritabanı bağlantısı ısıtılır.
- DB yoksa process kapanmaz; warning loglanır ve /api/ready 503 döndürmeye devam eder.
- Shutdown'ta engine.dispose() çağrılır.

Health endpoint'leri

Endpoint	Davranış	Kubernetes anlamı
GET /api/health	healthy, version, environment, uptime	Temel uygulama bilgisi
GET /api/live	Process/event loop canlıysa 200	Liveness: pod yeniden başlatma kararı
GET /api/ready	DB SELECT 1 başarılıysa 200, değilse 503	Readiness: trafik alıp almama kararı

Market data akışı

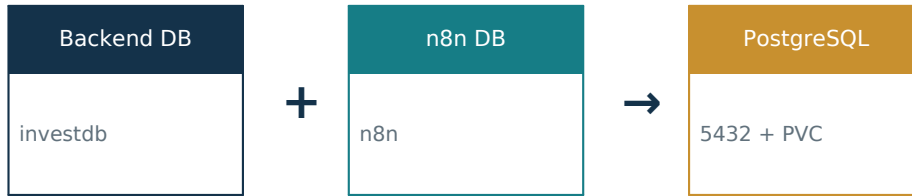
```
GET /api/markets/overview
→ get_market_provider()
→ MARKET_DATA_PROVIDER=yahoo ise YahooFinanceProvider
→ MARKET_DEFINITIONS içindeki semboller için quote()
→ MarketOverview(markets=[...])
```

```
GET /api/markets/{symbol}/history?interval=1d[]=1mo
→ provider.history() → MarketHistory(candles=[...])
```

Mevcut provider seçimi yalnızca yahoo ve mock davranışını uygular. values-dev.yaml'da provider yahoo görünse de .env.example varsayılanı mock'tur; hangi ortamın gerçekten çalıştığı pod env'ine bağlıdır. Yahoo çağrısı başarısız olursa overview tek bir sembol yüzünden bütünü düşürmez; history çağrısı ise 502 döndürür.

5. PostgreSQL ne işe yarar, nasıl çalışır?

PostgreSQL, platformun kalıcı veri katmanıdır. Bitnami PostgreSQL chart'ı StatefulSet benzeri kalıcı bir veritabanı kurulumu, Secret tabanlı kimlik bilgileri ve PVC ile disk sürekliliği sağlar. Backend asyncpg üzerinden postgresql+asyncpg:// bağlantı şemasını kullanır; n8n ise kendi native postgres sürücüsü nedeniyle postgres:///DB_* değişkenleriyle bağlanır.



- investdb backend uygulama verileri içindir: Phase 2+ modeller, fiyat serileri, haber, sinyal, portföy ve raporlar burada tutulabilir.
- n8n database workflow tanımları, execution metadata ve credential/config state içindir; backend iş veritabanından ayrıdır.
- primary.persistance.size dev'de 2Gi, prod'da 50Gi olarak override edilir; storageClass boşsa cluster'ın varsayılanı kullanılır.
- initdb script'i n8n database'ini oluşturur ve investuser'a yetki verir; bu işlem yalnızca ilk database initialization koşullarında etkilidir.

Önemli mevcut durum: Phase 1 market endpoint'leri Yahoo'dan okur ve response üretir, fakat bu kod yolunda market sonucunu PostgreSQL'e yazan repository/model katmanı henüz yoktur. PostgreSQL bağlantısının bugünkü gözle görünür kullanımı startup/readiness kontrolüdür; kalıcı finansal veri modeli sonraki faz işidir.

6. Env, ConfigMap ve Secret akışı

Ayarlar üç kaynaktan yönetilir: local development için .env, Kubernetes'te non-secret değerler için ConfigMap, secret değerler için Kubernetes Secret/SecretRef. Pydantic Settings değişken isimlerini case-insensitive olarak alan adlarına eşler; örneğin DATABASE_URL → database_url, MARKET_DATA_PROVIDER → market_data_provider.

Değişken grubu	Örnekler	Kubernetes'te kaynak
Backend davranışı	ENVIRONMENT, LOG_LEVEL, DEBUG, AI_PROVIDER, MARKET_DATA_PROVIDER, NEWS_PROVIDER	ConfigMap

Değişken grubu	Örnekler	Kubernetes'te kaynak
DB bağlantısı	DATABASE_URL; host/user/name Helm ile, password Secret ile	ConfigMap + SecretRef
API anahtarları	INTERNAL_API_KEY, OPENAI_API_KEY, MARKET_DATA_API_KEY, NEWS_API_KEY	SecretRef
n8n	DB_TYPE, DB_POSTGRESDB_HOST/DATABASE/USER, N8N_LOG_LEVEL	ConfigMap
n8n güvenliği	N8N_ENCRYPTION_KEY, DB_POSTGRESDB_PASSWORD	SecretRef
Frontend	VITE_API_BASE_URL build-time; BACKEND_HOST runtime	Vite env + Deployment env

Backend DATABASE_URL örneği
 postgresql+asyncpg://investuser:\${_DB_PASS}@:5432/investdb

Local .env örneği
 DATABASE_URL=postgresql+asyncpg://investuser:change-me@localhost:5432/investdb
 MARKET_DATA_PROVIDER=mock
 INTERNAL_API_KEY=change-me-internal-api-key

Helm Secret template stringData üretir; production için değerlerin --set, External Secrets Operator, Vault veya Sealed Secrets üzerinden sağlanması amaçlanmıştır. Gerçek secret'ların values.yaml, values-prod.yaml veya Git içindeki ArgoCD manifestlerinde tutulmaması gerekir.

7. Kubernetes ve Helm çalışma modeli

Helm chart, feature flag gibi çalışan enabled alanlarıyla bileşenleri koşullu üretir. Deployment'lar RollingUpdate kullanır; backend ve frontend için maxUnavailable=0 ayarıyla güncelleme sırasında erişilebilirlik korunmaya çalışılır. Backend ve n8n, PostgreSQL erişilebilir olana kadar busybox initContainer içinde nc ile bekler.

Kaynak	Rol
Deployment	Pod replica, image, env, securityContext, resource limit ve probe tanımları
Service	Cluster içi DNS ve port erişimi; backend/n8n ClusterIP, frontend varsayılan NodePort
ConfigMap	Secret olmayan backend/n8n ayarları ve service discovery URL'leri
Secret	DB/API/n8n encryption key gibi hassas değerler
PVC	PostgreSQL ve n8n state sürekliliği
Ingress	Production'da TLS ve host/path yönlendirmesi
NetworkPolicy	Production'da default deny + açıkça izin verilen bağlantılar
HPA	Tanımlı ancak values-prod.yaml'da disabled; replica sayısı sabit

Probe davranışı

- Backend startup/liveness /api/live, readiness /api/ready kullanır.
- Frontend /health, n8n /healthz ile kontrol edilir.
- Readiness başarısız backend pod'unu Service trafiğinden çıkarır; bu, database geçici olarak kapalıyken hatalı trafik dağıtımını azaltır.

SecurityContext'ler backend, frontend ve n8n için non-root kullanıcı, privilege escalation kapalı, capability drop ve service account token otomount kapalı gibi kısıtlar uygular. Backend read-only root filesystem kullandığı için /tmp emptyDir mount edilir; frontend nginx için cache ve pid dizinleri emptyDir ile yazılabilir hale getirilir.

8. ArgoCD ve GitOps akışı

ArgoCD Application kaynakları repository'nin main branch'ini izler. Her Application aynı chart yolunu farklı Helm parametreleriyle render eder; böylece bileşenler bağımsız sync edilir ve bir bileşenin değişimi diğerlerinin Deployment'ını gereksiz yere yeniden kurmaz.

Git push
→ ArgoCD repo refresh
→ Helm render (values.yaml + values-dev.yaml + parameters)
→ Kubernetes apply
→ automated sync / selfHeal / prune
→ Pod probes ve rollout durumu

Application	Açtığı bileşen	Kapattığı bileşenler
investment-platform-be	Backend	Frontend, n8n, embedded PostgreSQL
investment-platform-fe	Frontend	Backend, n8n, embedded PostgreSQL
investment-platform-n8n	n8n	Backend, frontend, embedded PostgreSQL
investment-platform-postgre	Bitnami PostgreSQL 18.8.6	Chart içindeki diğer bileşenler

Split-app modelinde backend, frontend ve n8n'in kendi chart release adlarından üretilen Service/Secret isimleri ile global.* değerlerinde yazan sabit isimler aynı olmalıdır. Bu sözleşme bozulursa pod çalışsa bile frontend backend'e, backend/n8n PostgreSQL'e DNS veya Secret üzerinden ulaşamayabilir.

9. n8n neden var ve ne yapacak?

n8n, finansal hesaplamaların yerine geçen bir servis değildir. Onun rolü “ne zaman çalışacak, hangi sistemi çağırarak, sonuç hangi karara gidecek?” sorularını görsel workflow'larla yönetmektir. Backend; veri çekme, validasyon, hesaplama ve domain kurallarının sahibi kalır. n8n ise bu fonksiyonları schedule, HTTP Request, IF, Code, AI, email ve webhook düğümleriyle birleştirir.

n8n kullanım alanı	Örnek akış
Zamanlama	Her sabah market verisi çekme ve analiz başlatma
Orkestrasyon	Backend endpoint'lerini sırayla çağırma; başarısız adımda branch
Dış veri	KAP, SEC, TCMB, FRED, ECB, haber ve veri sağlayıcıları
AI koordinasyonu	Yapılandırılmış backend çıktısını AI provider'a gönderme
Bildirim	Email, webhook veya ileride mesajlaşma kanallarına alarm gönderme
Operasyon	Health/readiness kontrolü, execution log ve retry

Repository'deki mevcut health-check workflow'u her 5 dakikada /api/health çağırır; status healthy ise healthy log, değilse unhealthy log branch'ine gider. Workflow export edilmiş ancak active:false durumundadır; canlı n8n'e import/activate edilmeden otomatik çalışmaz.

n8n persistence PVC ile /home/node/.n8n dizinini korur. Community Edition için Recreate strategy ve tek replica seçilmiştir; aynı PVC'ye iki aktif n8n pod'unun yazması engellenir. Production'da n8n dışarıya varsayılan olarak açılmaz; ingress.n8n.enabled false'tur.

10. Güvenlik ve işletim gözlemleri

Mevcut tasarımda güçlü taraflar

- Secret değerleri için Helm SecretRef, runtime env ve dış secret sistemi geçiş yolu tanımlanmış.
- Production NetworkPolicy default-deny ile başlıyor; backend, n8n, PostgreSQL ve ingress bağlantıları ayrı izinlerle sınırlandırılıyor.
- Pod'lar non-root çalışıyor, service account token otomatik bağlanmıyor ve container capability'leri düşürülüyor.
- TLS Ingress ve cert-manager annotation'ı production override'ında hazır.

Dikkat edilmesi gereken bulgular

- argocd/postgres-app.yaml içinde auth.password değeri düz metin olarak investpass yazılmış. Bu değer Git'e commit edilmemeli; existingSecret veya External Secrets yaklaşımı kullanılmalı ve mevcut parola döndürülmeli.
- values.yaml içindeki global backend/database/postgres secret isimleri split-app ArgoCD release isimleriyle uyumlu kontrol edilmeden deploy edilmemeli.
- values-prod.yaml AI_PROVIDER=openai, MARKET_DATA_PROVIDER=alpha_vantage ve NEWS_PROVIDER=newsapi seçiyor; ilgili API key Secret değerleri ve provider kodları tamamlanmadan production'da gerçek veri/AI beklentisi karşılanmayabilir.
- values-prod.yaml'da autoscaling yapılandırması tanımlı olsa da enabled:false; kaynak kullanımı artınca HPA kendiliğinden devreye girmez.
- NetworkPolicy dış provider'lara 80/443 izni veriyor; DNS, egress gözlemlenebilirliği ve provider rate-limit/retry politikaları ayrıca işletilmelidir.

11. Fazlar ve uygulamanın evrimi

Faz	Planlanan yetenek	Bugünkü dayanak
1	Kubernetes foundation, health, market overview	Mevcut ve çalışır temel
2	DB modelleri ve mock market data	SQLAlchemy Base ve Alembic altyapısı hazır
3	SMA, EMA, RSI, MACD, ATR, ADX, Bollinger, VWAP	Henüz eklenmedi
4	Fundamental analysis ve valuation	Henüz eklenmedi
5	News ve resmi açıklamalar	Provider ayarları var, iş akışı henüz yok
6	Signal/risk/market regime	Henüz eklenmedi
7	n8n daily analysis, news, ideas, portfolio workflows	Health-check örneği mevcut
8	AI analyst	Provider ayarı/mock seçeneği mevcut
9	Email ve alerts	n8n egress izinleri geleceğe hazırlıyor
10-11	Portfolio, watchlist, backtesting, AI performance	Henüz eklenmedi

Önerilen bir sonraki teknik adım

Önce split-app deployment sözleşmesini tekilleştirmek ve düz metin PostgreSQL secret'ını kaldırmak; ardından SQLAlchemy modelleri, Alembic migration'ları ve market data persistence akışını eklemek en düşük riskli sıradır. Bu temel tamamlanmadan analiz, AI ve n8n rapor workflow'larının üzerine çıkılması gözlemlenebilirlik ve veri sürekliliği açısından erken olur.

12. Kaynak ve doğrulama haritası

Konu	Kaynak dosyalar
Genel hedef mimari	README.md
Local env sözleşmesi	.env.example
Backend lifecycle/config	backend/app/main.py; backend/app/core/config.py
Health/readiness	backend/app/api/health.py
Market data	backend/app/api/markets.py; backend/app/services/market_data.py; backend/app/schemas/market.py
Frontend API ve dashboard	frontend/src/services/api.ts; frontend/src/pages/DashboardPage.tsx; frontend/nginx.conf
Base Helm değerleri	helm/investment-platform/values.yaml
Environment override	helm/investment-platform/values-dev.yaml; values-prod.yaml
Kubernetes wiring	helm/investment-platform/templates/*.yaml
GitOps	argocd/backend-app.yaml; frontend-app.yaml; n8n-app.yaml; postgres-app.yaml
n8n örnek workflow	n8n/workflows/health-check.json

Sonuç: Platformun omurgası doğru sorumluluk ayırımına sahip ve Kubernetes'e taşınabilir biçimde hazırlanmış durumda. Ancak mevcut davranış hâlâ temel gözlemlenebilirlik ve market görünümü seviyesinde; kalıcı finansal veri, analiz motoru, sinyal, AI ve bildirim değer zinciri sonraki implementasyonlarla oluşacaktır.